

• • • • • • • •  
• • • • • • • •  
• • • • • • • •



# W4 COS30045

Lecture

Class

• • • • • • • • •  
• • • • • • • • •

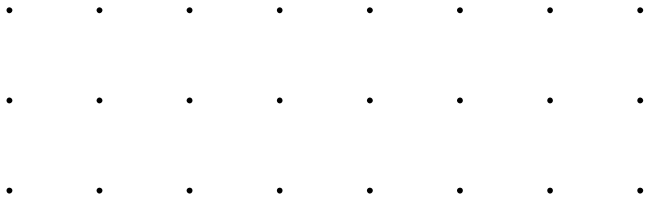
Agenda

- 4.1 DOM and SVG
- 4.2 Introduction to D3
- 4.3 Loading data and building charts
- 4.4 Scaling and labelling charts

Agenda

- Drawing SVGs
- Building bar chart with D3

• • • • • • • • •  
• • • • • • • • •  
• • • • • • • • •  
• • • • • • • • •  
• • • • • • • • •  
• • • • • • • • •



# W4.1 COS30045

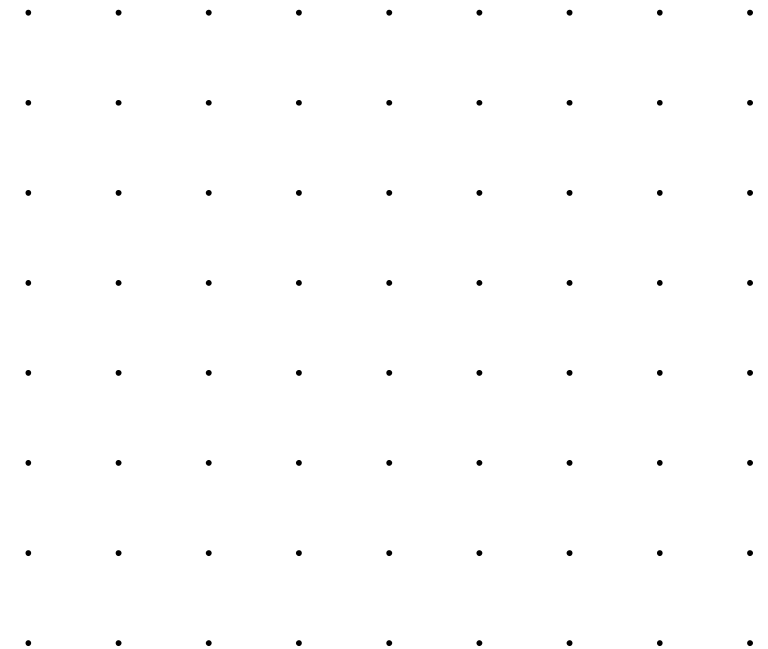
## Lecture

### Agenda

- DOM
- SVG

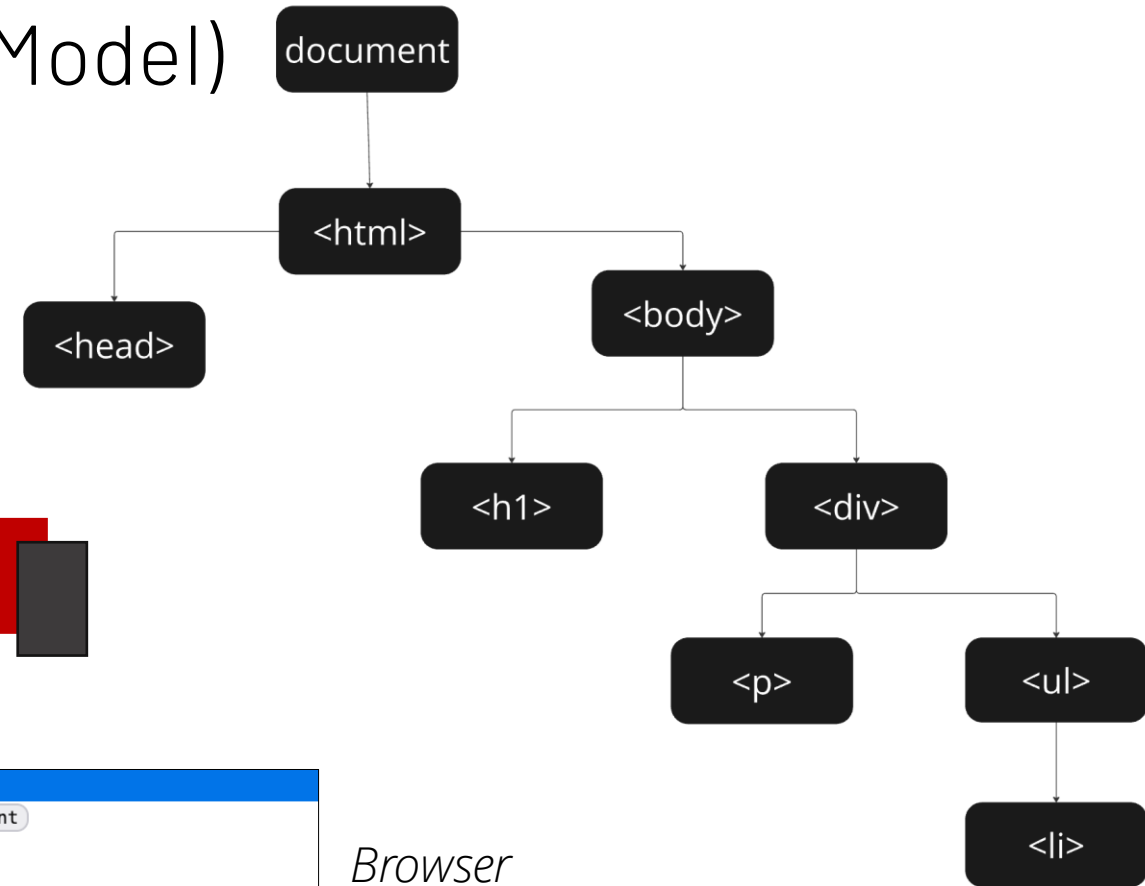
### Reference

- Dufour & Meeks (2024) Ch 1



# HTML and DOM (Document Object Model)

- Browser creates DOM from HTML
- DOM describes content of document
- Elements are nested
- Elements are rendered in the order they appear in DOM



Code

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>DOM Structure Example</title>
6  </head>
7  <body>
8    <h1>Welcome to the DOM Example</h1>
9    <div>
10     <p>This is a simple example.</p>
11     <ul>
12       <li>Item 1</li>
13       <li>Item 2</li>
14       <li>Item 3</li>
15     </ul>
16   </div>
17 </body>
18 </html>
```

DOM

```
<!DOCTYPE html>
<html lang="en"> event
  <head> ... </head>
  <body>
    <h1>Welcome to the DOM Example</h1>
    <div>
      <p>This is a simple example.</p>
      <ul>
        <li>...</li>
        <li>...</li>
        <li>...</li>
      </ul>
    </div>
  </body>
</html>
```

Browser

## Welcome to the DOM Example

This is a simple example.

- Item 1
- Item 2
- Item 3

- 
- 
-

# DOM

In DOM you can manipulate:

## Style

- Text styles
- Layout
- Colours and backgrounds
- Borders and outlines....

## Attributes

- Element identification (classes and ids)
- Hyperlinks (e.g., href)
- Images (e.g., src, alt)

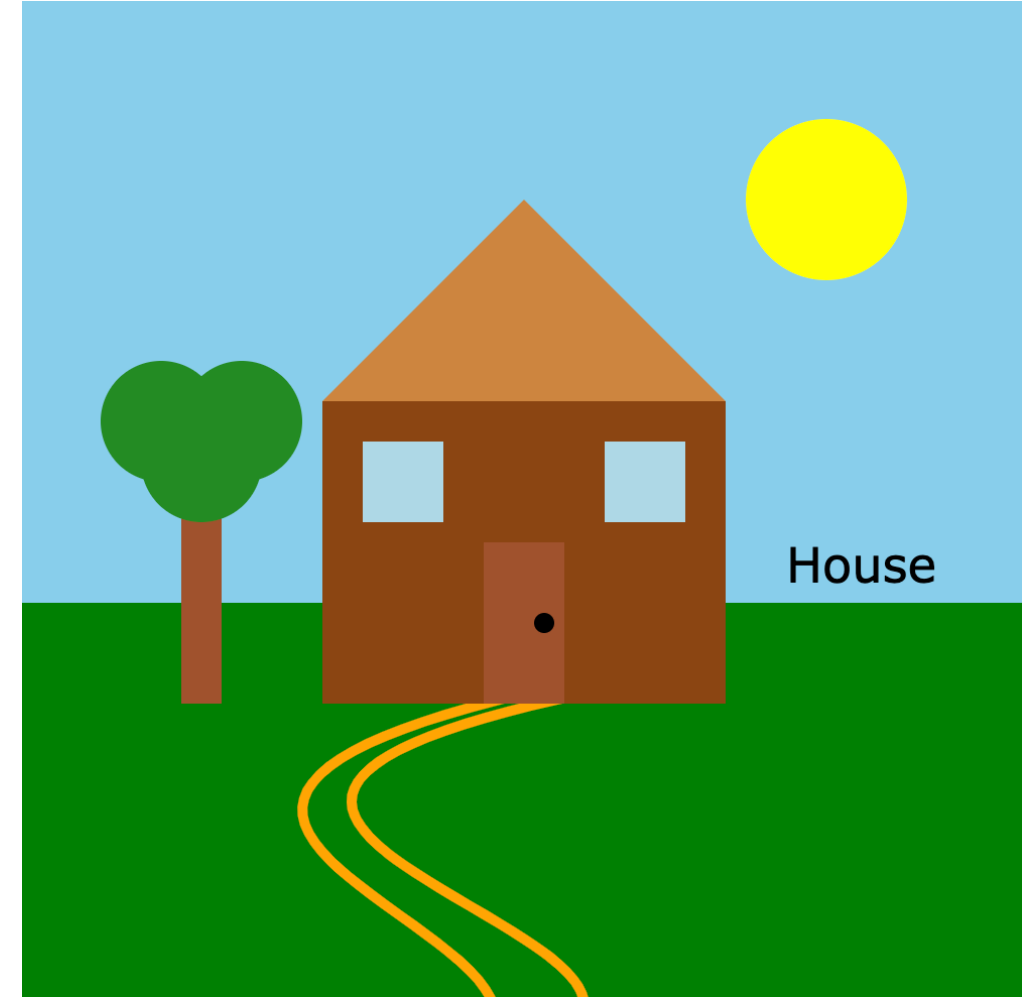
## Properties (dynamic)

- Element content (e.g., innerHTML)
- Attributes (e.g. setAttribute())
- Classes (e.g. className)
- Events (e.g., addEventListener())

. . . . .  
. . . . .

# Scalable Vector Graphics (SVG)

- Represented by mathematical expressions
- Scalable to any size without losing quality
- Often more efficient than raster-based formats (e.g., PNG, JPG)
- Can be added to the DOM and dynamically manipulated and styled with JavaScript (e.g., D3.js)
- Supports interactivity and animation
- Widely supported by modern web browsers



Edited GitHub Copilot code

# SVG container

All `svg` shapes contained within `svg` parent

```
<body>  
  <svg></svg>  
</body>
```

```
<body>  
  <svg width="900" height="300">  
</svg>
```

This code only generates the container  
Must add shapes onto canvas

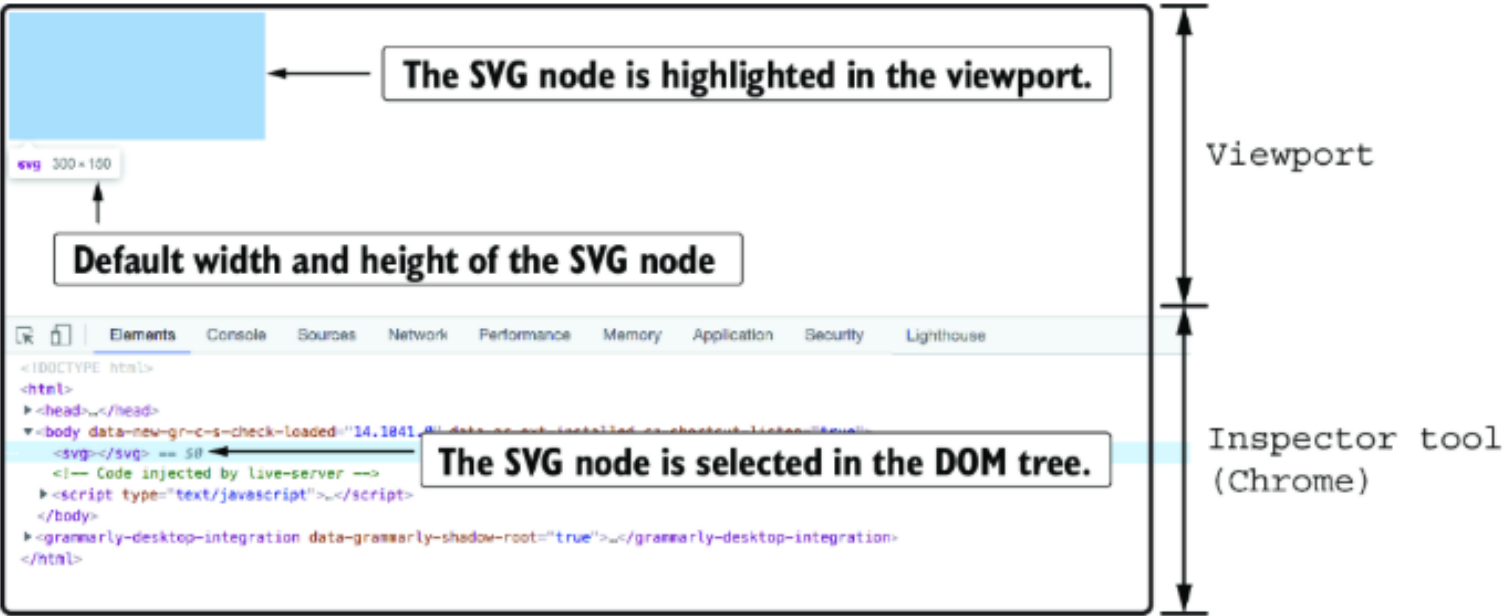


Figure 1.9 SVG node selected in the DOM tree and highlighted in the viewport

.....  
.....

# SVG container

Manipulate size

```
<body>  
  <svg width="900" height="300">  
  </body>
```

Add boarder

```
<body>  
  <svg width="900" height="300" style="border:1px solid black;">  
  </body>
```

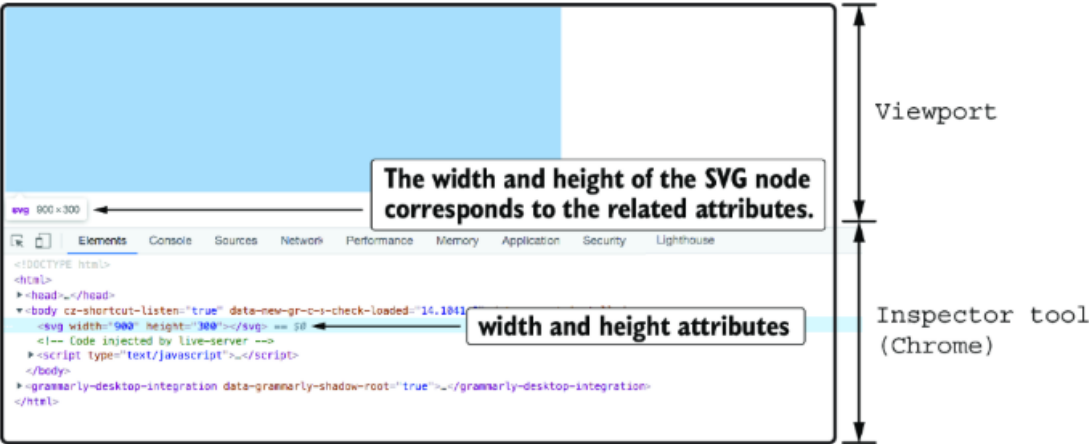
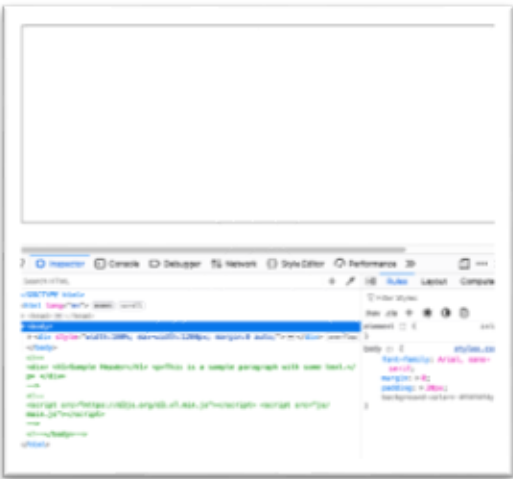
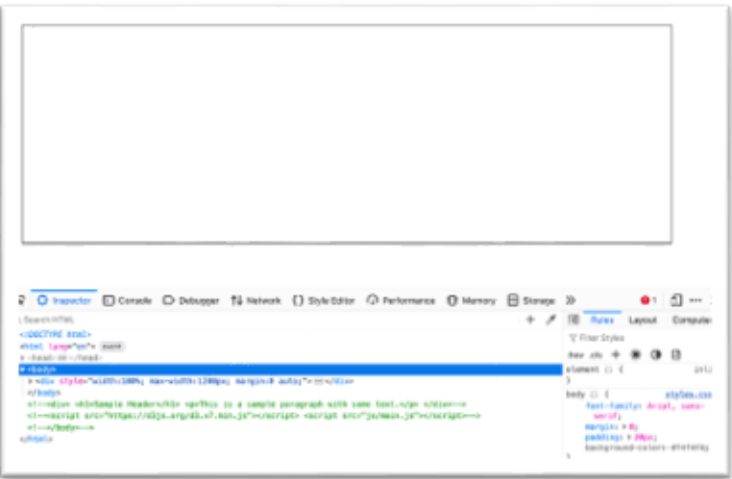


Figure 1.10 SVG node taking the size specified by its attributes



# SVG container

Make responsive

```
<body>
```

```
  <svg viewBox="0 0 900 300" style="border:1px solid black;"></svg>
```

```
</body>
```

Can change position on window

Keep relative to other content on page

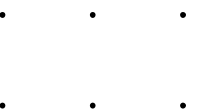
```
<body>
```

```
  <div style="width:100%; max-width:1200px; margin:0 auto;">
```

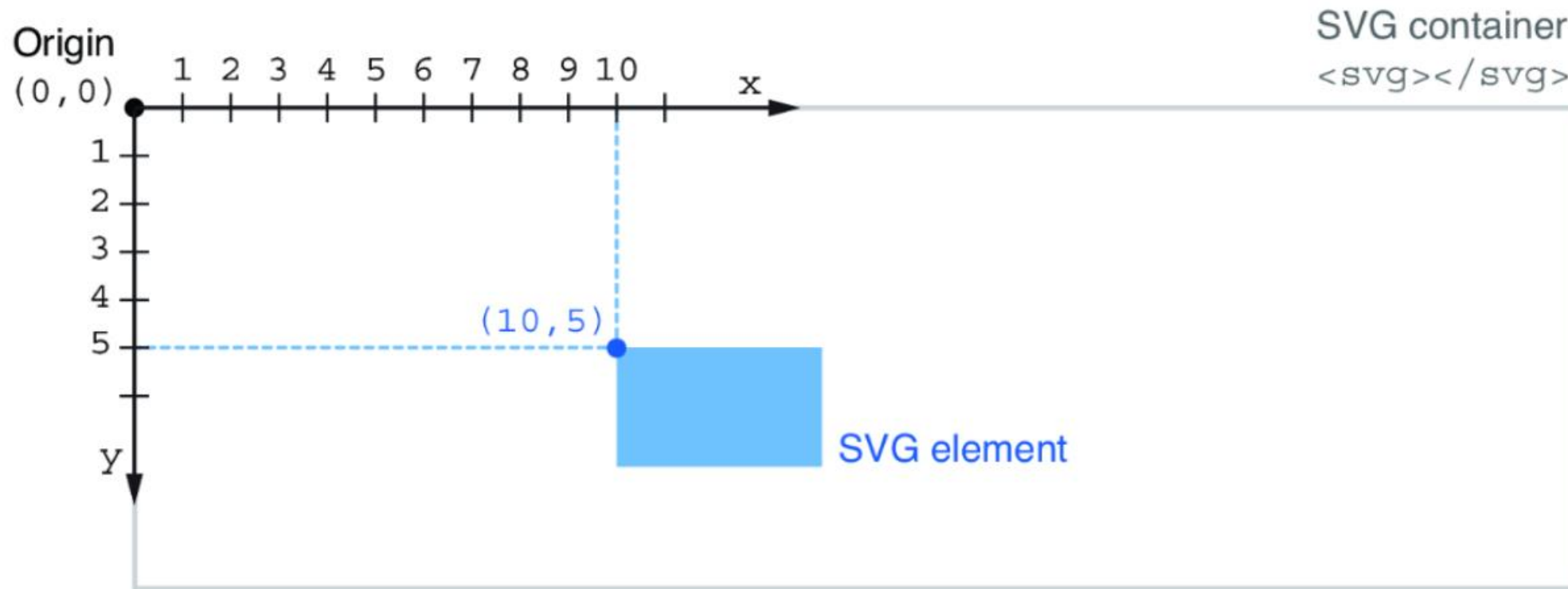
```
    <svg viewBox="0 0 900 300" style="border:1px solid black;"> </svg>
```

```
  </div>
```

```
</body>
```



# SVG coordinate system

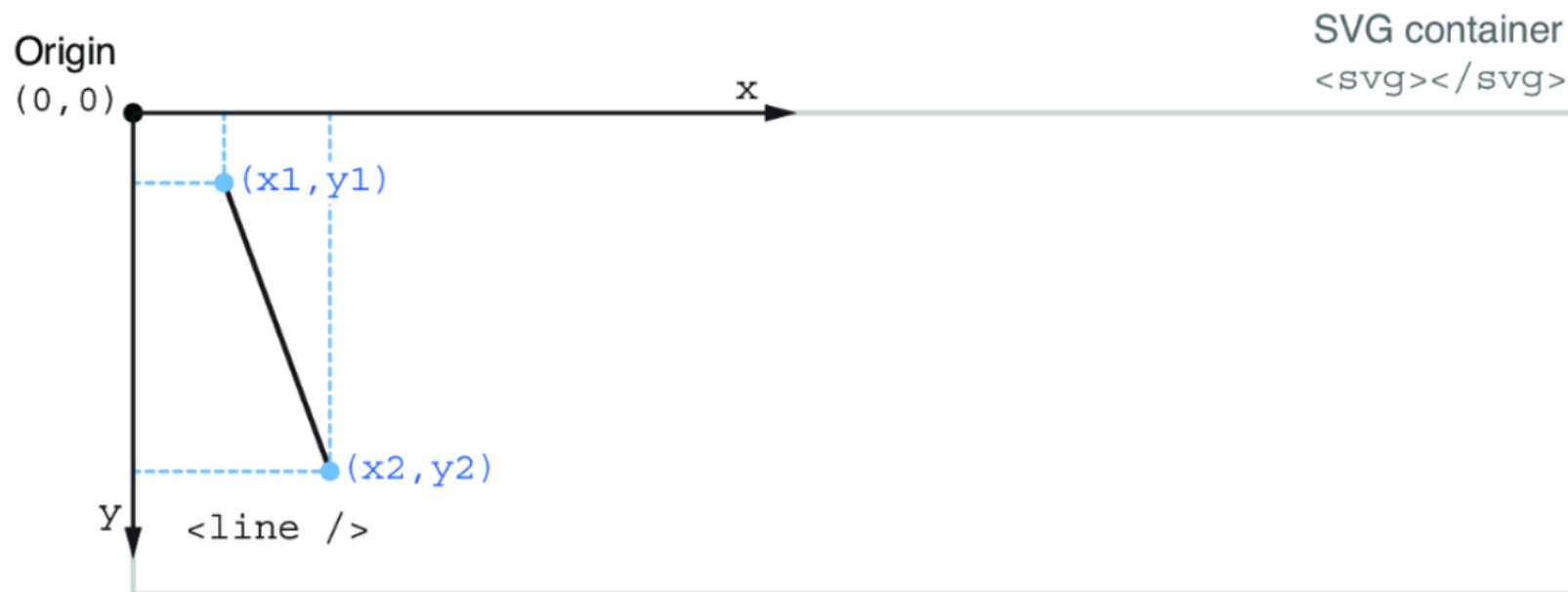


**Figure 1.11 Coordinate system of the SVG container and position of an element**

. . . .  
. . . .

# SVG coordinate system

```
<svg>  
  <line x1="50" y1="45" x2="140" y2="225" />  
</svg>
```



**Figure 1.12 Positioning a line element in the coordinate system of an SVG container**

# SVG Shapes for Visualisation

## Line

```
<line x1="-20" y1="45" x2="140" y2="225" stroke="black" />
```

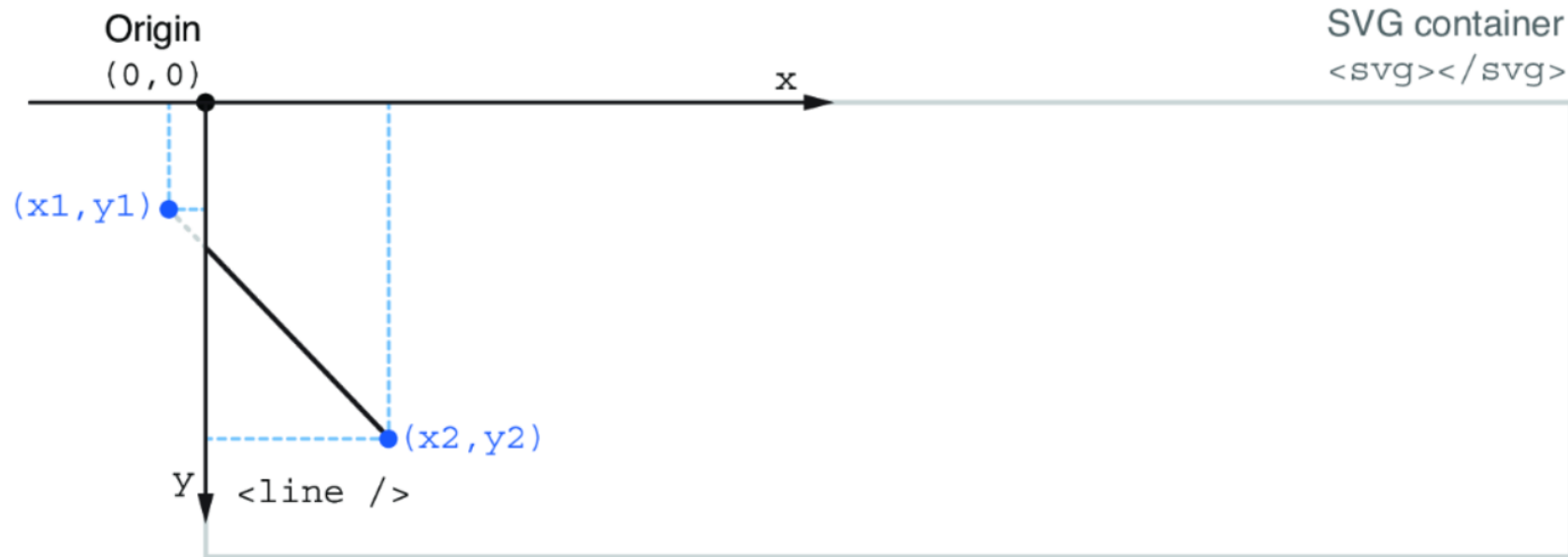
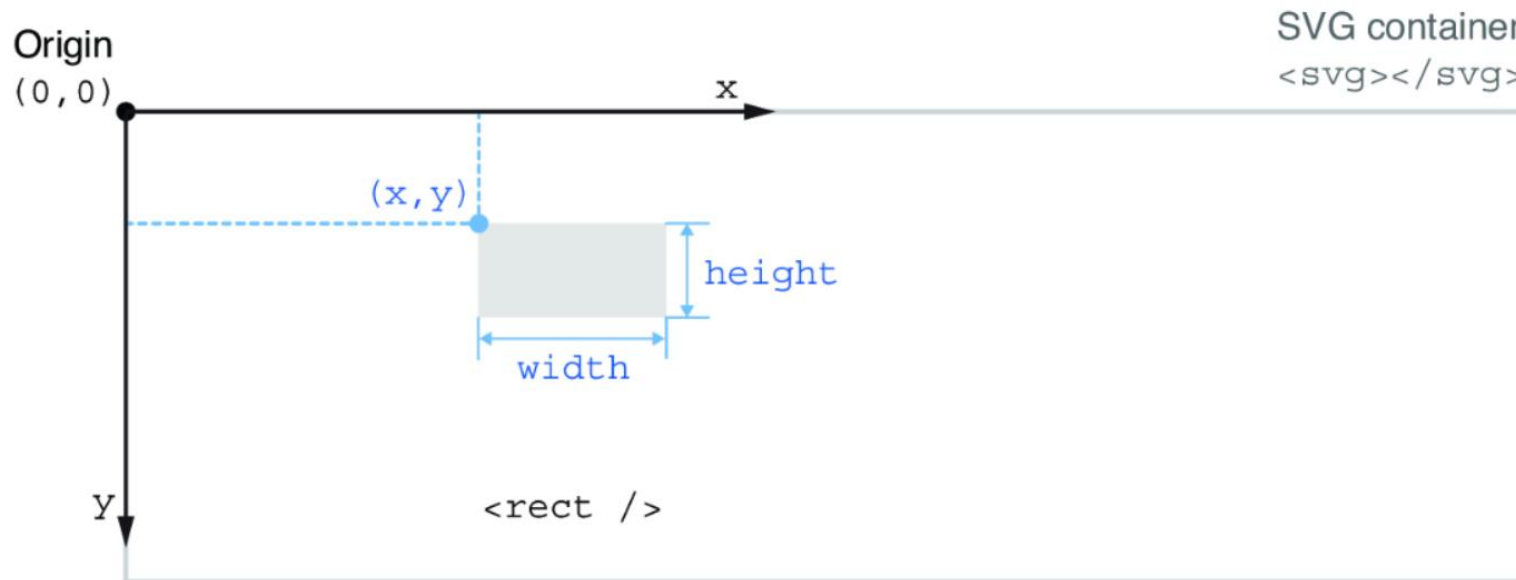


Figure 1.13 SVG line partially hidden when outside of the SVG container

# SVG Shapes for Visualisation

## Rectangle

```
<rect x="260" y="25" width="120" height="60" fill="#6ba5d7" />
```



**Figure 1.14 Positioning and sizing a rectangle in the coordinate system of an SVG container**

# SVG Shapes for Visualisation

## Formatting shapes

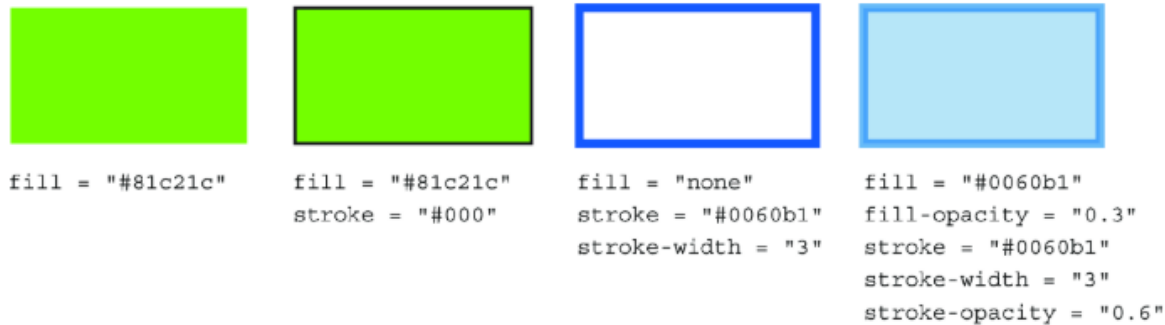
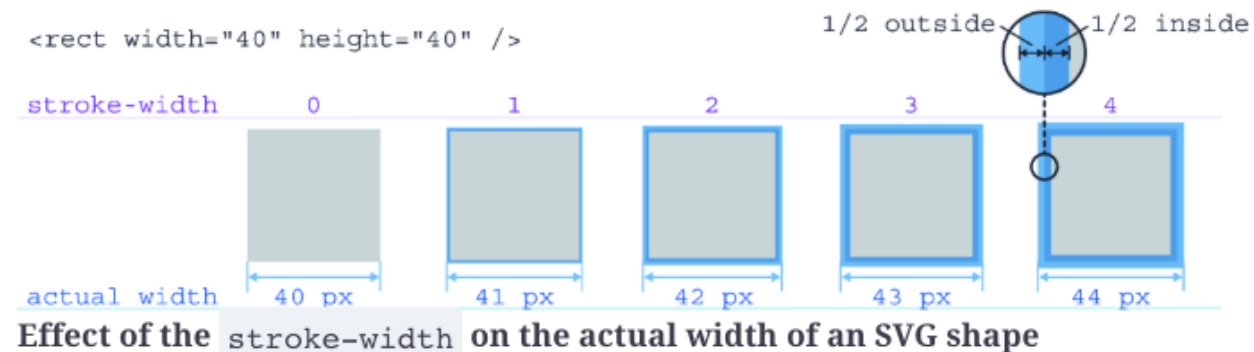


Figure 1.15 Different styling attributes applied to rectangle SVG shapes

## Aligning shapes



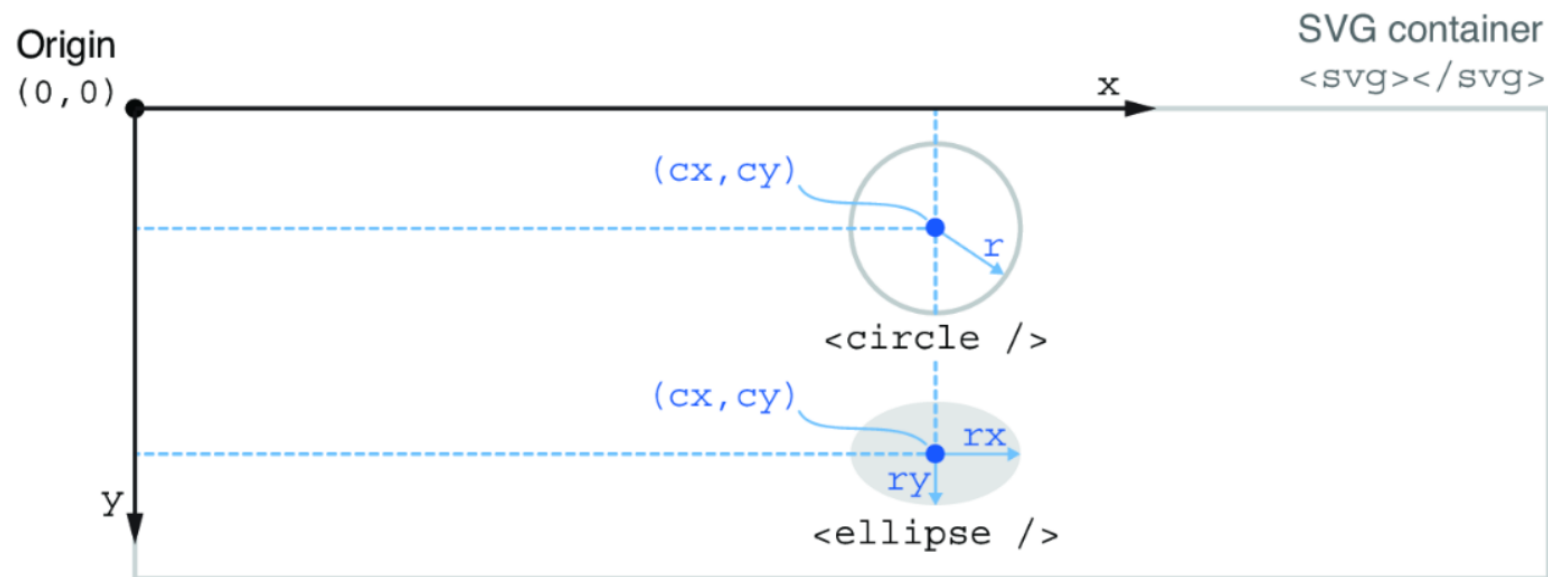
# SVG Shapes for Visualisation

## Circle

```
<circle cx="530" cy="80" r="50" fill="none" stroke="#81c21c" stroke-width="3" />
```

## Ellipse

```
<ellipse cx="530" cy="205" rx="50" ry="30" fill="#81c21c" />
```



**Figure 1.17 Positioning and sizing a circle and an ellipse in the coordinate system of an SVG container**

# SVG Shapes for Visualisation

|      |                        |                                       |                        |
|------|------------------------|---------------------------------------|------------------------|
| Path | Move (M)<br>coordinate | Cubic Curve (C)<br>Steepness of curve | Stop (S)<br>coordinate |
|------|------------------------|---------------------------------------|------------------------|

```
<path d="M680 150 C 710 80, 725 80, 755 150 S 810 220, 840 150" fill="none" stroke="#773b9a" stroke-width="3" />
```

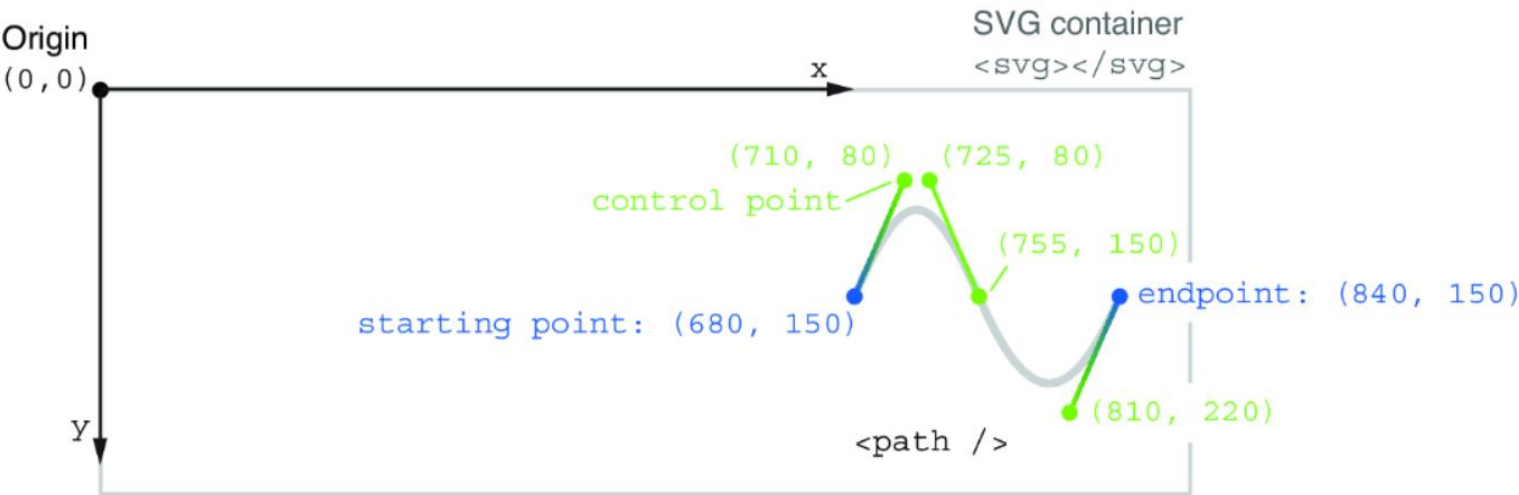
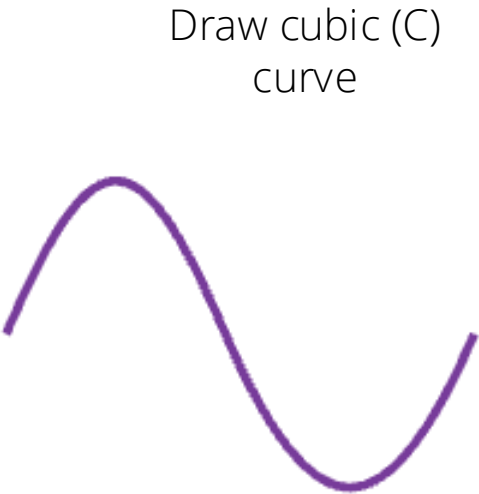


Figure 1.18 A simple SVG path and its control points

# SVG Shapes for Visualisation

## Text

`<text>SVG text</text>`

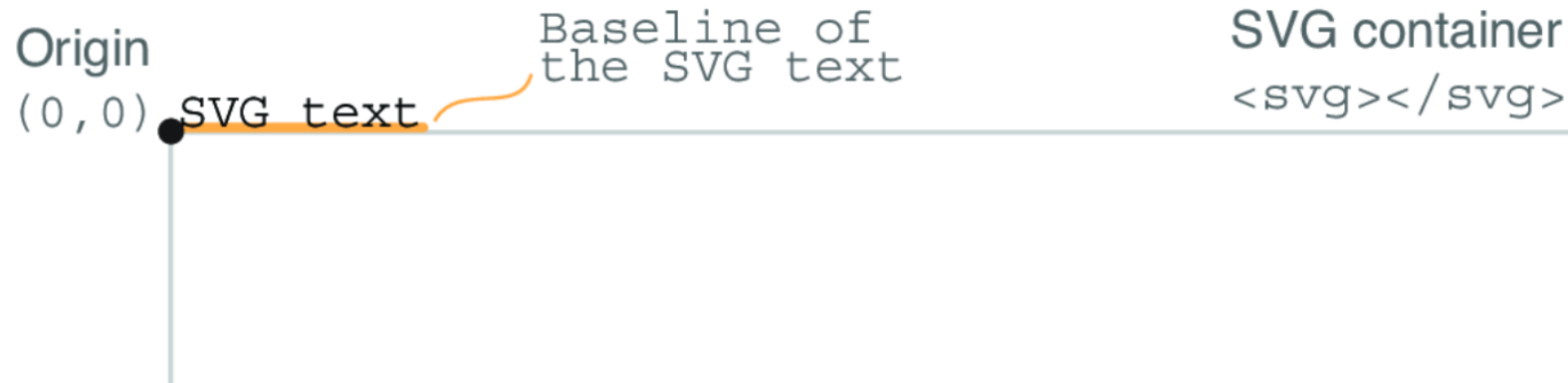


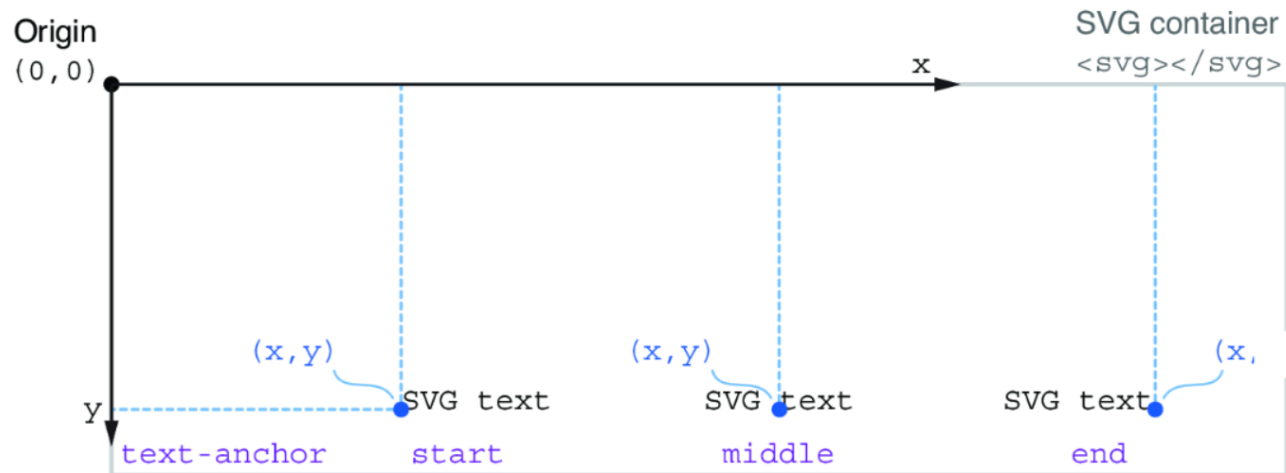
Figure 1.19 Text positioned outside of the SVG container

`<text x="0" y="0" font-family="Verdana" font-size="24" fill="black">SVG text</text>`

# SVG Shapes for Visualisation

## Text

```
<text x="380" y="155" text-anchor="middle">circle</text>
```



**Figure 1.20** The `text-anchor` attribute affects the alignment of SVG text. Its default value is `start`. To align a text element based on its middle, we apply a `text-anchor` attribute of `middle`. Similarly, to align a text based on its end, we apply a `text-anchor` attribute of `end`.

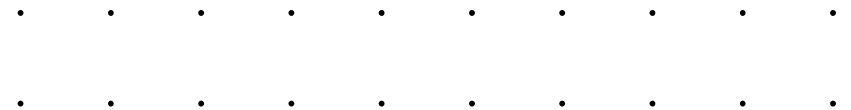
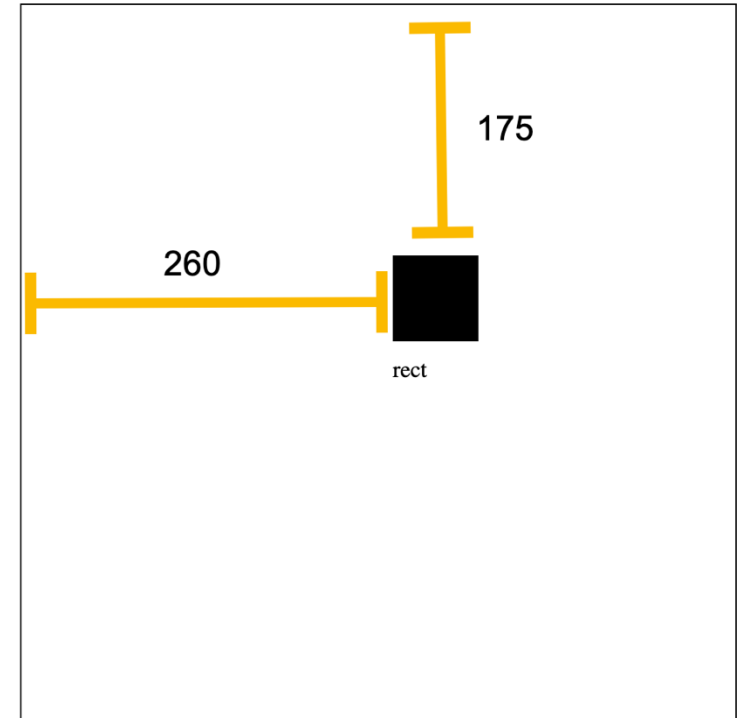
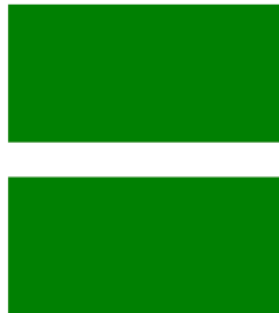
. . . . .  
. . . . .

# SVG Shapes for Visualisation

## Grouping

```
<g transform="translate(260,175)">  
  <rect x="0" y="0" width="60" height="60" />  
  <text x="0" y="85" text-anchor="start">rect</text>  
</g>
```

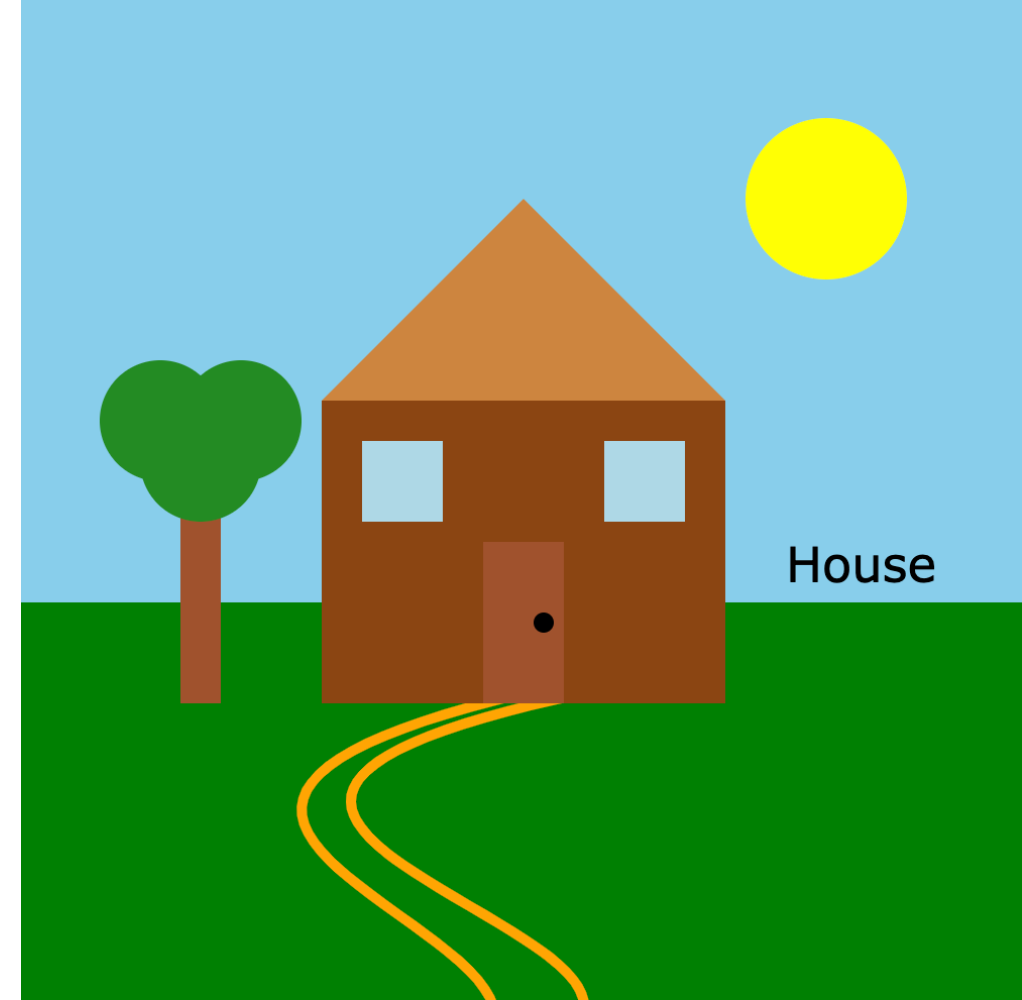
```
<g fill="green">  
  <rect x="260" y="25" width="120" height="60"/>  
  <rect x="260" y="100" width="120" height="60"/>  
</g>
```



# Exercise 4.1 SVGs

1. Ask GitHub Copilot to build you a SVG house and garden
2. Include examples of:
  1. Shapes (e.g. line, rectangle circle and ellipses)
  2. Paths
  3. Text
  4. Grouping and transformations
  5. Styling
3. ....

This is **NOT** the HARD bit!  
Don't spend all class doing this!  
Do before you come to class!!



Edited GitHub Copilot code

Drawing does not need to look exactly like this, customise to show understanding of SVGs

You are not required use GitHub Copilot to do this task, but it may take you longer.